
Clasificador de dígitos con red neuronal sobre una cadena de FMAs en punto fijo

Proyecto final — Diseño de Unidades de Ejecución

Miguel Ángel Bucio

Maestría en Aritmética Digital

Especificación técnica del proyecto

Julio 2026

Contents

1	Objetivo y motivación	2
2	Fundamentos mínimos	2
2.1	Neurona	2
2.2	ReLU (Rectified Linear Unit)	2
2.3	Capa <i>fully connected</i>	3
2.4	Argmax	3
3	Arquitectura de la red y cuantización	3
3.1	Topología	3
3.2	Cuantización entera	4
3.3	El detalle clave: la requantización con SHIFT1 = 5	4
3.4	La capa de salida no lleva ReLU ni shift	5
4	Arquitectura de hardware	5
4.1	Interface RTL obligatoria	5
4.2	Grados de libertad del diseño	6
4.3	Bloques comunes a cualquier implementación	6
4.4	Opción A — Un datapath reutilizado para ambas capas	7
4.5	Opción B — Dos datapaths en pipeline entre imágenes	7
4.6	Ancho de la cadena de FMAs (N)	7
4.7	Estilo interno de acumulación	8
4.8	Layout de las memorias (compartido por cualquier opción)	10
4.9	FSM controladora	11

1 Objetivo y motivación

Este documento describe el diseño de un acelerador RTL que clasifica dígitos manuscritos (0–9) usando un *perceptrón multicapa* (MLP) totalmente conectado, implementado sobre una cadena de ocho unidades *fused multiply-add* (FMA) trabajando en aritmética entera. El objetivo pedagógico del proyecto es cerrar el ciclo entre lo que un ingeniero de unidades de ejecución estudia en clase y una aplicación real: la inferencia de una red neuronal es en $\approx 99\%$ operaciones *multiply-accumulate*, por lo que diseñar una NN inference engine es fundamentalmente diseñar una **cadena de FMAs bien orquestada**.

La analogía mental útil es la de *una TPU de una sola fila*: la unidad sistólica más chica capaz de calcular un producto punto. Al mantener el hardware minimalista (una fila de 8 celdas, no una malla 2D) y la red suficientemente pequeña, la implementación cabe en EDA Playground y sirve como laboratorio para discutir cuantización, precisión numérica, ciclos por inferencia, pipelining, hazards y verificación *bit-exact*.

Cifras clave (medidas sobre el modelo Python, no estimadas)

- **Accuracy en punto flotante** (360 imágenes de test): **97.22%**.
- **Accuracy con pesos y activaciones cuantizados a int8** (mismo test): **97.22%** — pérdida por cuantización **nula**.
- **Accuracy sobre las 50 imágenes del testbench** (subconjunto que se carga en EDA Playground): **94.0%** (47/50). *Esta es la cifra exacta que el RTL debe reproducir, independientemente de la arquitectura de hardware elegida.*

2 Fundamentos mínimos

Esta sección establece el vocabulario mínimo de redes neuronales para lectores con formación en diseño digital pero sin fondo en *machine learning*. Todo lo que se necesita para este proyecto entra en cuatro conceptos.

2.1 Neurona

Una neurona es una *función afín* seguida de una no linealidad. Para un vector de entrada $\mathbf{x} \in \mathbb{R}^N$, un vector de pesos $\mathbf{w} \in \mathbb{R}^N$ y un bias $b \in \mathbb{R}$:

$$y = f\left(\sum_{i=0}^{N-1} w_i x_i + b\right).$$

El núcleo del cálculo es el **producto punto** $\sum w_i x_i$, exactamente lo que hace una cadena de FMAs (**a*b + c**) realimentada.

2.2 ReLU (Rectified Linear Unit)

Es la no linealidad más común en redes modernas:

$$\text{ReLU}(x) = \max(0, x).$$

En hardware, sobre un valor *en complemento a dos*, ReLU se implementa con un *mux gobernado por el bit de signo*: si el signo es 1 la salida es 0, si es 0 pasa el valor completo.

2.3 Capa *fully connected*

Una capa densa con M neuronas de entrada y N de salida es una multiplicación matriz–vector:

$$\mathbf{y}_{N \times 1} = f(W_{N \times M} \mathbf{x}_{M \times 1} + \mathbf{b}_{N \times 1}).$$

Cada neurona de salida es independiente de las otras, así que se pueden calcular *una a la vez* en un hardware *time-multiplexed* y reutilizar los mismos multiplicadores.

2.4 Argmax

Al final, para clasificar, se elige el índice con el score más alto:

$$\hat{y} = \arg \max_i s_i.$$

No se necesita **softmax**: como es una función *estrictamente monótona*, el índice del máximo de las probabilidades y el índice del máximo de los scores son iguales. La red se ahorra el exponencial.

3 Arquitectura de la red y cuantización

3.1 Topología

La red es un MLP con una única capa oculta:

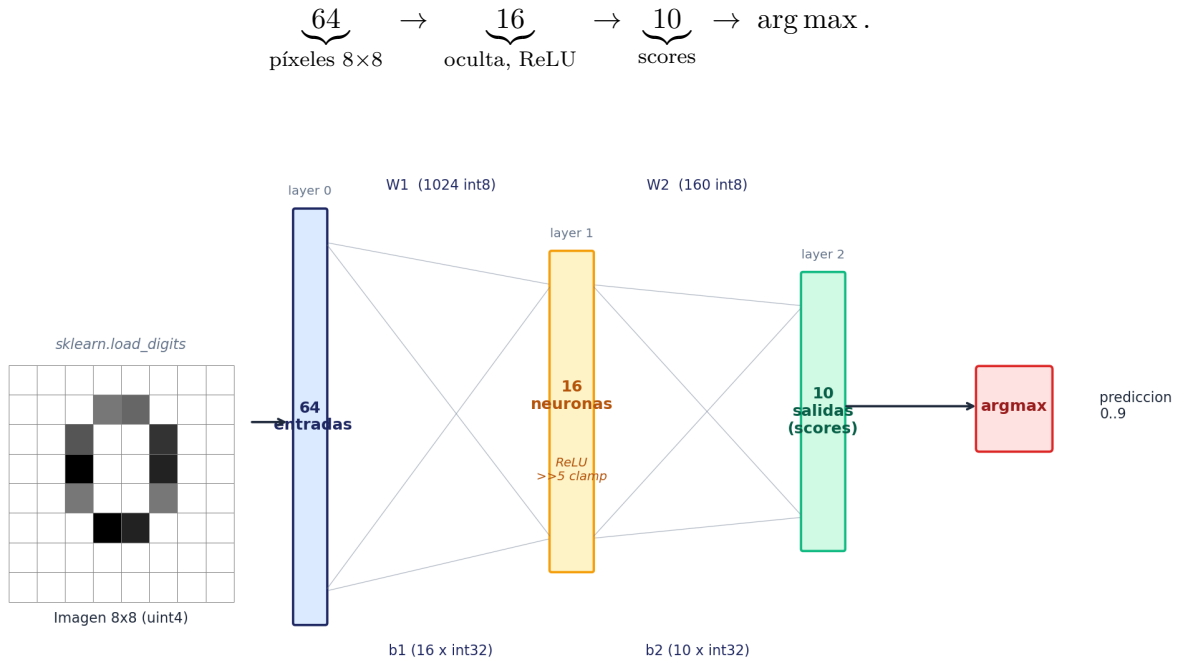


Figure 1: Topología del MLP $64 \rightarrow 16 \rightarrow 10$. La imagen viene de `sklearn.datasets.load_digits()` en resolución 8×8 con píxeles en el rango $[0, 16]$. Cada línea gris representa uno de los 1,184 pesos.

Dataset. Se usa `digits` de `scikit-learn` (`sklearn.datasets.load_digits`): 1,797 imágenes 8×8 con píxeles enteros en $[0, 16]$. **No es MNIST** (28×28); se eligió precisamente porque la resolución pequeña hace que el dataset completo y los pesos quepan en la memoria del simulador de EDA Playground. Split estratificado 80/20 con semilla `random_state=7`: 1,437 imágenes de entrenamiento y 360 de test.

3.2 Cuantización entera

Toda la aritmética *en el RTL* es entera. La conversión desde el modelo flotante hacia el modelo entero sigue una *cuantización simétrica por capa*:

Señal	Formato	Notas
Píxel de entrada	uint4	se recorta $\min(px, 15)$; el valor 16 se aplasta a 15
Pesos capa 1 y 2	int8	escala simétrica $s = 127 / \max w $
Bias capa 1	int32	$b_1^q = \text{round}(b_1 \cdot s_1)$
Bias capa 2	int32	$b_2^q = \text{round}(b_2 \cdot s_2 \cdot s_h)$
Producto de una celda FMA	int16	rango máximo $ w \cdot x \leq 127 \cdot 15$
Acumulador	int32	valor máximo medido: 5,520 — 12 bits, sobra
Activación oculta	uint8	tras $\text{ReLU} + \gg 5 + \text{clamp } 255$
Score final	int32	crudo, sin ReLU ni shift

Table 1: Formatos numéricos del datapath. Todas las señales están dimensionadas con margen sobre los valores extremos medidos en el conjunto de entrenamiento.

Escalas medidas. Sobre los pesos entrenados:

$$s_1 = \frac{127}{\max |W_1|} = 148.79, \quad s_2 = \frac{127}{\max |W_2|} = 139.42.$$

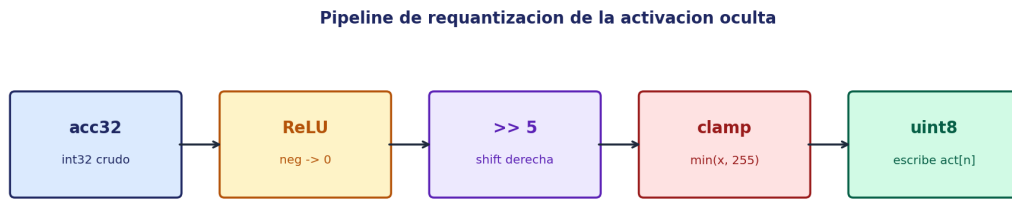
La escala de la activación oculta cuantizada a entero es

$$s_h = s_1 \cdot 2^{-\text{SHIFT1}} = s_1 / 32.$$

El bias de la segunda capa se cuantiza con $s_2 \cdot s_h$ para que todos los scores s_i compartan la misma escala; como el **argmax** solo requiere escala común (no valor absoluto), esa es toda la “desnormalización” que hace falta.

3.3 El detalle clave: la requantización con SHIFT1 = 5

Cuando la capa 1 termina de acumular sus 64 productos, el registro contiene un **int32** del orden de miles. Convertirlo directamente a **uint8** saturando a 255 aplastaría casi todos los valores al máximo y perdería información. La receta correcta, la que este diseño implementa, es un *clipped ReLU*: primero ReLU, luego un corrimiento aritmético a la derecha por una constante, luego *clamp* a $[0, 255]$.



SHIFT1 = 5 elegido con el percentil 99.9 de las pre-activaciones (equivale a ReLU con clip por saturación, tipo ReLU6)

Figure 2: Pipeline de requantización de la activación oculta. El shift por constante es cableado en hardware — no cuesta área ni tiempo.

Elección de SHIFT1 = 5. El shift se elige *offline* en el script de entrenamiento a partir del **percentil 99.9** de las pre-activaciones sobre el conjunto de entrenamiento:

$$\text{SHIFT1} = \left\lceil \log_2 \left(\frac{p_{99.9}(\text{acc32})}{255} \right) \right\rceil = 5.$$

Con $\gg 5$ (equivalente a dividir por 32) el 99.9% de las activaciones caen en $[0, 255]$ y solo el 0.1% restante se satura en 255. Esto es exactamente el mismo truco que usa ReLU6 en MobileNet: cortar el rango para permitir aritmética de baja precisión sin degradar el accuracy final.

3.4 La capa de salida no lleva ReLU ni shift

Los 10 scores de la capa de salida van *directo* al **argmax** como **int32** con signo. El comparador se inicializa con `best_score = 32'h8000_0000` (el mínimo **int32** representable), la comparación es *signed*, y en caso de empate gana el primer índice recorrido. Ese comportamiento es determinista y reproducible, por lo que sirve como *golden reference* bit-exacta contra el modelo Python.

4 Arquitectura de hardware

Esta sección presenta el **espacio de diseño** en el que se enmarca el proyecto. La red neuronal y su cuantización quedan fijas (Secciones 3 y 5), pero la implementación RTL tiene varias decisiones abiertas: cada alumno resuelve según sus prioridades de área, *throughput*, latencia y frecuencia. Cualquier implementación correcta reproducirá los mismos 47/50 aciertos de la referencia dorada.

4.1 Interface RTL obligatoria

Independientemente de las decisiones internas de diseño, el módulo top-level de todos los alumnos debe exponer **exactamente** la siguiente interface SystemVerilog. El testbench común se conecta contra estos puertos y nombres; cualquier desviación rompe la verificación automática.

```

1 module neural_network_digits(
2     input  logic          clk,
3     input  logic          rst,
4     input  logic          start,
5     input  logic [IMAGE_PIXEL_WIDTH-1:0] image [IMAGE_HORIZONTAL_SIZE-1:0]
6                                     [IMAGE_VERTICAL_SIZE-1:0], // 8
6                                     x8 Digit
7     output logic          done,
8     output logic [DIGIT_WIDTH-1:0] digit
9 );
```

Parámetros fijos.

Parámetro	Valor	Descripción
IMAGE_PIXEL_WIDTH	4	bits por píxel (uint4, 0..15)
IMAGE_HORIZONTAL_SIZE	8	columnas de la imagen
IMAGE_VERTICAL_SIZE	8	filas de la imagen
DIGIT_WIDTH	5	bits de la salida (rango 0..9, sobra un bit)

Semántica de los puertos de control.

- **clk, rst**: reloj y *reset*. El *reset* es **activo en alto**; el testbench lo mantiene en 1 durante los primeros ciclos y luego lo baja a 0 antes de emitir el primer **start**.
- **image[H][V]**: matriz 8×8 de píxeles **uint4**. Debe estar estable *antes* del pulso de **start** y puede cambiar solo después de que **done** suba.
- **start**: **pulso de un ciclo** que indica que hay un nuevo dígito a procesar. El módulo comienza el cómputo en el flanco siguiente y no vuelve a mirar **start** hasta terminar.

- **done:** pulso de un ciclo que sube cuando el resultado está listo. En el mismo ciclo, **digit** contiene la predicción válida. Fuera de ese ciclo, **done** vale 0 (el testbench solo captura el flanco de subida).
- **digit:** predicción final (0..9) sobre 5 bits. Su valor solo es válido cuando **done** está alto; entre inferencias puede quedar en cualquier valor sin afectar la verificación.

Diagrama de tiempos. La secuencia esperada por el testbench para procesar una imagen es:

Ciclo	Acción
0	el TB pone image con la k -ésima imagen
1	el TB levanta start durante <i>un</i> ciclo
2 ... $T-1$	el DUT procesa; done = 0
T	el DUT levanta done durante <i>un</i> ciclo; digit válido
$T+1$	el TB registra el resultado y prepara la imagen $k+1$

El número de ciclos T entre **start** y **done** es libre — lo fija la arquitectura interna elegida por cada alumno.

4.2 Grados de libertad del diseño

Las tres decisiones principales son ortogonales entre sí:

1. **Reutilización de capas.** Usar el *mismo* datapath para ambas capas cambiando las fuentes con muxes (opción A, *time-multiplexed*), o instanciar *dos* datapaths dedicados que trabajen en pipeline entre imágenes (opción B).
2. **Ancho de la cadena de FMAs (N).** Cuántas celdas **multiply-accumulate** procesan un producto punto en paralelo. No hay ningún valor obligatorio; $N \in \{1, 2, 4, 8, 16, 32, 64\}$ son opciones razonables con distinto compromiso área/ciclos.
3. **Combinacional o pipelined.** Sin registros dentro de la cadena (camino crítico largo, control trivial) o con registros entre celdas (*throughput* de 1 por ciclo, pero aparecen *read-after-write hazards* sobre el acumulador realimentado).

4.3 Bloques comunes a cualquier implementación

Independientemente de las decisiones anteriores, todo diseño necesita los mismos componentes básicos:

- **ROM de pesos** $1,184 \times 8$ bits, cargada con `$readmemh("weights.hex", ...)`.
- **ROM de biases** 26×32 bits, desde `biases.hex`.
- **RAM / ROM de imagen** 64×4 bits, cargada por imagen desde `test_images.hex`.
- **Memoria de activaciones** de la capa oculta 16×8 bits, o registros *on-the-fly* si se hace pipeline entre capas (opción B).
- **Bloque de requantización** $\text{ReLU} \rightarrow \gg 5 \rightarrow \text{clamp } 255$. **No es negociable:** es parte del algoritmo entero y su omisión rompe la equivalencia con la referencia dorada.
- **Argmax al vuelo** con comparador *signed* inicializado en `32'h8000_0000`.
- **FSM controladora**, con contadores anidados de pasada, neurona y capa.
- **Una o más unidades FMA** (`fma_unit: out = a · b + c`), instanciadas con `generate` en la cantidad que el alumno elija.

4.4 Opción A — Un datapath reutilizado para ambas capas

Es la implementación con menor área. Un solo conjunto de FMAs, muxes controlados por la FSM que cambian las fuentes entre capa 1 y capa 2:

- **Capa 1 (64→16):** 16 neuronas $\times \lceil 64/N \rceil$ pasadas cada una.
- **Capa 2 (16→10):** 10 neuronas $\times \lceil 16/N \rceil$ pasadas cada una.
- Ciclos totales aproximados $\approx 16\lceil 64/N \rceil + 10\lceil 16/N \rceil$ (más overhead constante de *load* y *done*).

Con $N=8$ la aritmética da $16 \cdot 8 + 10 \cdot 2 = 148$ ciclos; con $N=16$ baja a $16 \cdot 4 + 10 \cdot 1 = 74$; con $N=64$ una neurona completa cabe en una pasada y bajan a $16 + 10 \cdot \lceil 16/64 \rceil = 26$; con $N=1$ (un solo multiplicador escalar) suben a $16 \cdot 64 + 10 \cdot 16 = 1184$.

4.5 Opción B — Dos datapaths en pipeline entre imágenes

Dos etapas hardware dedicadas, cada una con su propia cadena de FMAs y su tamaño propio (N_1 para la capa 1, N_2 para la capa 2). Mientras la etapa 2 procesa la imagen k , la etapa 1 ya está trabajando en la $k+1$. La memoria de activaciones se reduce a un par de *ping-pong* de 16×8 para desacoplar productor y consumidor.

- **Ventaja:** *throughput* aproximadamente el doble que la opción A a la misma frecuencia (mientras cada etapa trabaje cerca de su tope).
- **Ventaja adicional:** cada etapa puede dimensionarse por separado. La capa 1 hace 1,024 MACs, la capa 2 hace 160 — si se quieren tiempos parecidos por etapa conviene $N_1 \approx 6 \cdot N_2$.
- **Desventaja:** más área (dos memorias de pesos accedidas en paralelo, o una memoria dual-port; dos FSMs cooperantes).
- **Desventaja:** para el *primer* resultado la latencia es la suma de ambas etapas; el ahorro es en el estado estacionario.

4.6 Ancho de la cadena de FMAs (N)

El ancho es el botón de compromiso *área vs ciclos*. Ninguna elección es *la* correcta; algunas referencias:

N	Ciclos por inferencia (opción A)	Comentario
1	1,184 ciclos	un solo multiplicador; área mínima
2	≈ 592	compromiso agresivo
4	≈ 296	un mult + 4 sumas de camino crítico
8	148	el ejemplo mostrado en Fig. 3
16	≈ 74	la capa 2 ya cabe en 1 pasada por neurona
64	≈ 26	una neurona completa por pasada; área máxima

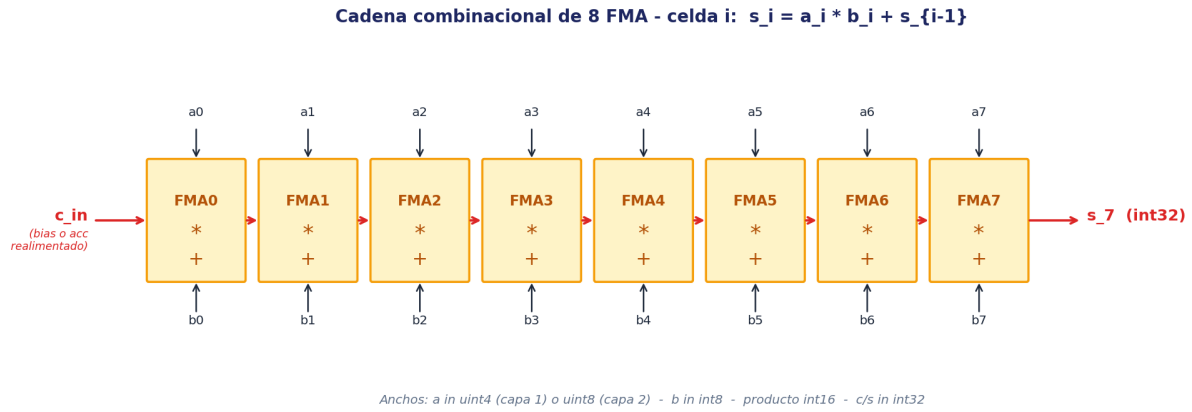


Figure 3: Cadena de $N=8$ FMAs como ejemplo de implementación. La entrada c_in del primer FMA recibe el bias en la primera pasada o el acumulador realimentado en las siguientes. La misma estructura sirve para N arbitrario — basta con parametrizar el `generate`. Estructuralmente idéntica a la fila básica de un arreglo sistólico o a un filtro FIR de N taps.

4.7 Estilo interno de acumulación

La tercera decisión se aplica *dentro* del bloque que resuelve una pasada del producto punto. Tres estilos son razonables:

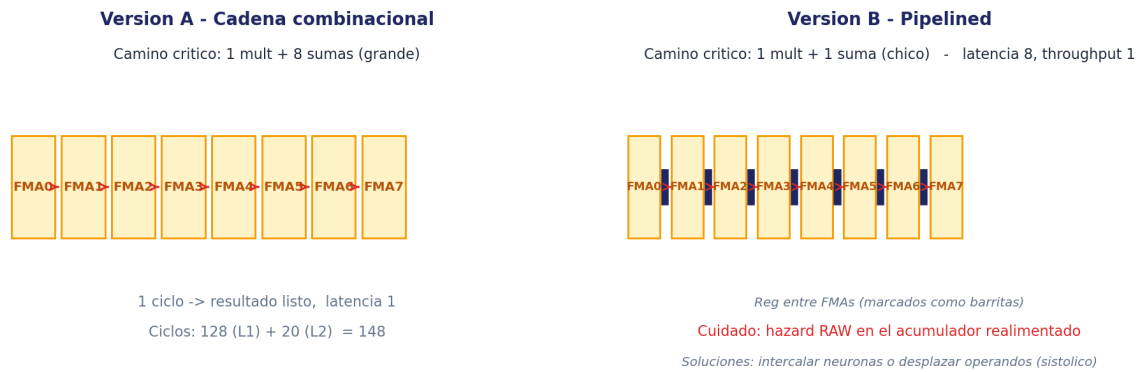


Figure 4: Variantes A y B de la cadena serial. La versión A es combinacional (camino crítico grande). La versión B intercala registros entre celdas para acortar el camino crítico a 1 mult + 1 suma, a costa de latencia N y hazards en el acumulador.

Variante A — Cadena combinacional serial.

- Camino crítico: *una multiplicación + N sumas en cadena.*
- Latencia: 1 ciclo por pasada; el resultado está listo al final del ciclo y se puede realimentar.
- Control trivial: no hay hazards, no hay *bubbles*.
- Desventaja: la frecuencia máxima cae cuando N crece. Para $N = 8$ y operandos `int16/int32`, un mult + 8 sumas satura los 15–20 ns típicos en síntesis para FPGA con LUTs de 4 entradas.

Variante B — Cadena *pipelined*.

- Camino crítico: *una multiplicación + una suma*, independiente de N .

- Latencia: N ciclos (una etapa por celda); *throughput* de 1 resultado por ciclo.
- Frecuencia máxima crece considerablemente porque el camino crítico colapsa al tamaño de una celda.
- Costo: registros entre celdas, control más elaborado, y aparición de *hazards*.

El hazard RAW en el acumulador realimentado. En la variante A el acumulador se lee y se escribe en el mismo ciclo; el resultado de la pasada k está disponible a tiempo para ser c_in de la pasada $k+1$. En la variante B el acumulador aparece en la salida N ciclos después de haber consumido su valor previo; emitir la pasada $k+1$ inmediatamente produciría una lectura con el valor *viejo* del acumulador. Dos soluciones clásicas de diseño de unidades de ejecución funcionan aquí:

1. **Intercalar neuronas en vuelo** (*latency hiding*). En lugar de terminar todas las pasadas de una neurona antes de empezar la siguiente, se emite la pasada 0 de N neuronas, luego la pasada 1 de todas, etc. Cada acumulador ya completó su vuelta antes de ser reusado. Costo: N acumuladores en paralelo, pero el pipeline se rellena al 100%.
2. **Escalonar los operandos con registros** (*transformación sistólica*). Se agregan retrasos crecientes a las entradas a y b de las celdas $1..N-1$ para alinear el resultado del vuelo previo con el c_in del vuelo actual. Es el mismo truco con el que se construye un array sistólico 2D a partir de una malla combinacional.

Variante C — Multiplicadores en paralelo con árbol de CSAs. En lugar de encadenar *multiplicar-y-sumar* serialmente, se disparan los N multiplicadores **en paralelo** (todos consumen su pareja (a_i, b_i) en el mismo ciclo) y sus N productos parciales se suman con un árbol de *Carry-Save Adders* (compresores 3:2 o 4:2). Es la misma idea de un árbol de Wallace/Dadda que se usa en multiplicadores de alto rendimiento, aplicada aquí a la reducción de un producto punto.

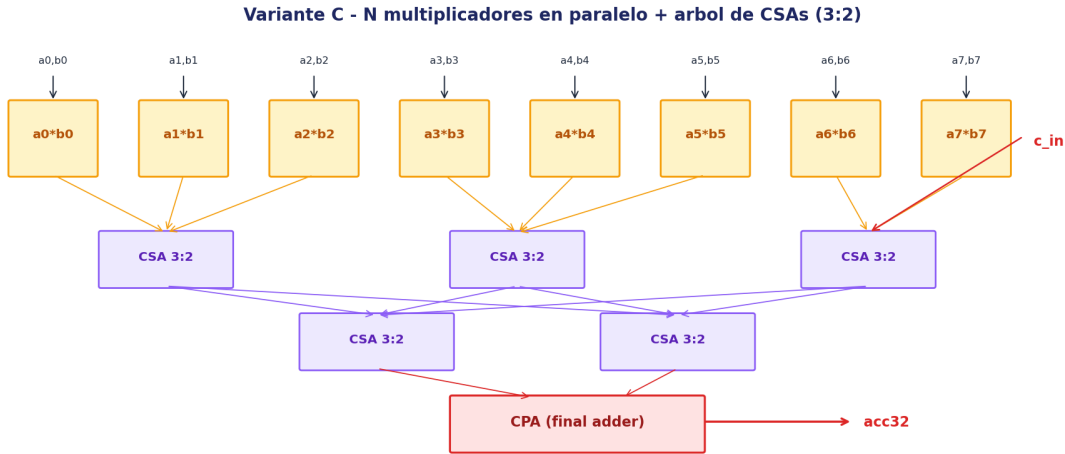
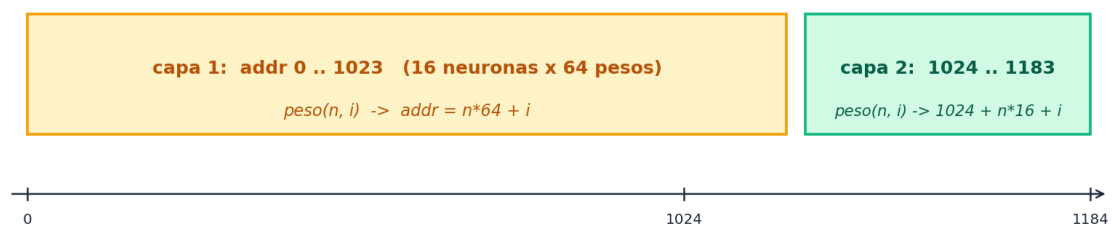


Figure 5: Variante C con $N = 8$: los ocho multiplicadores corren en paralelo y sus productos + el acumulador entran a un árbol de compresores 3:2. Los CSAs no propagan carry, así que su retardo es constante e independiente de la anchura. Solo el último nivel es un *Carry-Propagate Adder* (CPA) que convierte el par redundante (*sum*, *carry*) al binario final.

- Camino crítico: *una multiplicación + $\mathcal{O}(\log N)$ niveles de CSA + un CPA final*. Para $N = 8$ eso es típicamente **1 mult + 3 CSAs + 1 CPA** — del mismo orden que un solo FMA *pipelined*, pero con *todos* los productos ya sumados en una sola pasada.

- #### 4.8 Layout de las memorias (compartido por cualquier opción)



peso capa 2: $\text{addr} = 1024 + 16n + i, \quad n \in [0, 9], i \in [0, 15].$

4.9 FSM controladora

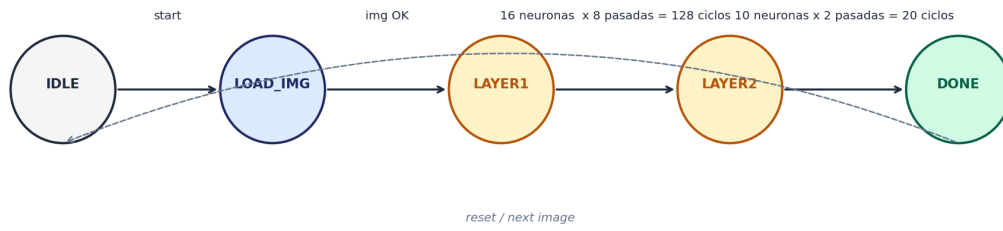


Figure 7: Máquina de estados finita a nivel de bloque para la opción A. Contadores anidados de *pasada*, *neurona* y *capa* coordinan los accesos a memoria, la selección de muxes y la validez del *writeback*. En la opción B cada etapa tiene su propia FSM cooperante.